Selection sort

```
def selection_sort(arr):
    """
    Sorts a list of elements using the selection sort algorithm.
    """
    n = len(arr)
    # Traverse through all array elements
    for i in range(n):
        # Find the minimum element in the remaining unsorted array
        min_indx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_indx]:
                min_indx = j

        # Swap the minimum element with the current element (arr[i])
        arr[i], arr[min_indx] = arr[min_indx], arr[i]
    return arr

# Example usage:
my_list = [64, 25, 12, 22, 11]
print(f"Original list: {my_list}")

sorted_list = selection_sort(my_list)
print(f"Sorted list: {sorted_list}")
```

Insertion sort

```
arr = [5, 2, 9, 1, 5, 6]


print("Original array:", arr)


for i in range(1, len(arr)):
    key = arr[i]
    j = i - 1


    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j -= 1


    arr[j + 1] = key


print("Sorted array:", arr)
```

MERGE SORT

```
def merge_sort(a):
    if len(a) <= 1:
        return a


    # Split the list into two halves
    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
```

```
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    return result + left[i:] + right[j:]

print(merge_sort([5, 2, 8, 1, 3]))
```

Selection Sort using C++

```
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {64, 25, 12, 22, 11};
    int n = 5;

    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
```

```
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }

        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }

    cout << "sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

Insertion Sort using C++

```
#include <iostream>
using namespace std;

int main() {
int arr[5] = {5, 2, 4, 6, 1};
int n = 5;
```

```
for (int i = 1; i < n; i++) {
int key = arr[i];
int j = i - 1;

while (j >= 0 && arr[j] > key) {
arr[j + 1] = arr[j];
j--;
}

arr[j + 1] = key;
}

cout << "sorted array: ";
for (int i = 0; i < n; i++) {
cout << arr[i] << " ";
}

return 0;
}
```

Merge Sort using C++

```
#include <iostream>
using namespace std;

void merge(int a[], int l, int m, int r) {
   int i = l, j = m + 1, k = 0;
   int temp[100];

   while (i <= m && j <= r) {
      if (a[i] < a[j])
         temp[k++] = a[i++];
      else
         temp[k++] = a[j++];
   }
```

```
    while (i <= m)
        temp[k++] = a[i++];

    while (j <= r)
        temp[k++] = a[j++];

    for (i = l, k = 0; i <= r; i++, k++)
        a[i] = temp[k];
}

void mergesort(int a[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;
        mergesort(a, l, m);
        mergesort(a, m + 1, r);
        merge(a, l, m, r);
    }
}

int main() {
    int a[] = {5, 2, 8, 1, 3};
    int n = 5;

    mergesort(a, 0, n - 1);

    for (int i = 0; i < n; i++)
        cout << a[i] << " ";

    return 0;
}
```

Knapsack Problem (0/1) using C++

```
#include <iostream>
using namespace std;
```

```
int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n, W;
    cout << "enter no of items: ";
    cin >> n;

    int wt[n], val[n];

    cout << "enter weights: \n";
    for (int i = 0; i < n; i++)
        cin >> wt[i];

    cout << "enter value: \n";
    for (int i = 0; i < n; i++)
        cin >> val[i];

    cout << "enter Capacity of knapsack: ";
    cin >> W;

    int dp[n + 1][W + 1];

    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (wt[i - 1] <= w)
                dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]],
                               dp[i - 1][w]);
            else
                dp[i][w] = dp[i - 1][w];
        }
```

```
    }

    cout << "Maximum profit: " << dp[n][W];

    return 0;
}
```

---

5. Travelling Salesman Problem (TSP) using C++

```
#include <iostream>
#include <climits>
using namespace std;

int n;
int dist[10][10];
int dp[1 << 10][10];

int tsp(int mask, int pos) {
    if (mask == (1 << n) - 1)
        return dist[pos][0];

    if (dp[mask][pos] != -1)
        return dp[mask][pos];

    int ans = INT_MAX;

    for (int city = 0; city < n; city++) {
        if ((mask & (1 << city)) == 0) {
            int newAns = dist[pos][city] +
                         tsp(mask | (1 << city), city);
            ans = min(ans, newAns);
        }
    }

    return dp[mask][pos] = ans;
```

```
}

int main() {
    cout << "enter number of cities: ";
    cin >> n;

    cout << "enter distance matrix: \n";
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            cin >> dist[i][j];

    for (int i = 0; i < (1 << n); i++)
        for (int j = 0; j < n; j++)
            dp[i][j] = -1;

    cout << "Minimum travel cost: " << tsp(1, 0);

    return 0;
}
```

Binary Search using C++

```
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {2, 4, 6, 8, 10};
    int key, low = 0, high = 4, mid;

    cout << "enter number to search: ";
    cin >> key;

    while (low <= high) {
        mid = (low + high) / 2;

        if (arr[mid] == key) {
```

```
        cout << "element found at position " << mid;
        return 0;
    }
    else if (arr[mid] < key) {
        low = mid + 1;
    }
    else {
        high = mid - 1;
    }
  }

  cout << "Element not found";
  return 0;
}
```

---

2. Sequential / Linear Search using C++

```
#include <iostream>
using namespace std;

int main() {
  int n, key;

  cout << "Enter the number of elements: ";
  cin >> n;

  int a[n];

  cout << "enter elements: ";
  for (int i = 0; i < n; i++)
    cin >> a[i];

  cout << "enter element to search: ";
  cin >> key;
```

```
    for (int i = 0; i < n; i++) {
        if (a[i] == key) {
            cout << "Element found at position " << i + 1;
            return 0;
        }
    }

    cout << "Element not found";
    return 0;
}
```